

EduTrack System Documentation — Part 1

Backend to Database Connection, Schema Mapping & Mechanics Guide

1. Executive Overview & Database Architecture

The EduTrack backend is constructed using Python 3 and the **FastAPI** framework. All data persistence, entity mapping, and relational integrity are managed via **SQLAlchemy 2.0**, a powerful Object-Relational Mapper (ORM). The underlying database engine defaults to **SQLite** (stored locally at `backend/edutrack.db`), while maintaining seamless compatibility with production relational engines such as PostgreSQL.

Key Architectural Concept: SQLAlchemy acts as an abstraction bridge between Python objects in memory and raw database tables. Instead of writing manual SQL queries, the application executes structured Python function calls which SQLAlchemy translates into optimized SQL statements.

2. Engine Configuration & Session Lifecycle

Database connections are managed inside `backend/app/database/session.py`. The connection lifecycle follows a strict factory pattern to prevent memory leaks and thread safety issues:

```
# backend/app/database/session.py
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker
from app.config import settings

# 1. Create Engine with thread safety rules for SQLite
connect_args = {}
if settings.DATABASE_URL.startswith('sqlite'):
    connect_args['check_same_thread'] = False

engine = create_engine(settings.DATABASE_URL, connect_args=connect_args, pool_pre_ping=True)

# 2. Session Factory
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)

# 3. Dependency Injection Generator Function
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

Session Lifecycle Mechanics:

- **Engine Creation:** Initializes database connection pooling. `pool_pre_ping=True` checks connection health before issuing queries.
- **SessionLocal Factory:** Produces fresh Session instances for incoming HTTP requests. `autocommit=False` guarantees transaction safety.

- **get_db() Generator:** FastAPI injects db: Session = Depends(get_db) into router endpoints. The yield hands the active DB session to the handler, and the finally block ensures db.close() is invoked even if an exception occurs.

3. Declarative Base & Complete Entity Relationship Schema

All ORM models inherit from a common Base class defined in backend/app/database/base.py. Upon server boot (in main.py), Base.metadata.create_all(bind=engine) inspects all registered models and creates the database tables automatically if they do not exist.

Complete Database Models Mapping Matrix:

Entity Model	Table Name	Primary / Foreign Keys	Core Attributes	Cascade / Relationships
State	states	id (PK)	name (Unique)	institutions (1:N via District)
District	districts	id (PK), state_id (FK->states.id)	name	state (N:1), institutions (1:N, Cascade Delete)
InstitutionType	institution_types	id (PK)	name, category	institutions (1:N, Cascade Delete)
Institution	institutions	id (PK), district_id (FK), type_id (FK)	name, address, created_at	district, institution_type, courses (1:N), events (1:N)
Course	courses	id (PK), institution_id (FK)	name, code, duration_years	institution (N:1), branches (1:N, Cascade Delete)
Branch	branches	id (PK), course_id (FK)	name	course (N:1), academic_years (1:N, Cascade Delete)
AcademicYear	academic_years	id (PK), branch_id (FK)	year_label, total_seats, student_strength	branch (N:1), syllabus_entries (1:N, Cascade Delete)
SyllabusEntry	syllabus_entries	id (PK), academic_year_id (FK)	title, content_text, file_url	academic_year (N:1)
Event	events	id (PK), institution_id (FK)	title, event_date, description	institution (N:1)
InformationPost	information_posts	id (PK), institution_id (FK)	title, body, category	institution (N:1)
User	users	id (PK)	username (Unique), email (Unique), hashed_password, role	System Auth User (Viewer, Editor, Admin)

4. Data Access Layer (CRUD Operations)

Database interactions are encapsulated inside specialized files in backend/app/crud/. This decouples HTTP router logic from raw database query construction.

Standard CRUD Query Patterns:

```
# Example pattern from backend/app/crud/institution_crud.py
from sqlalchemy.orm import Session
from app.models.institution import Institution
from app.schemas.institution import InstitutionCreate

# Read Operation with Filtering and Pagination
def get_institutions(db: Session, skip: int = 0, limit: int = 100, district_id: int = None):
    query = db.query(Institution)
    if district_id:
        query = query.filter(Institution.district_id == district_id)
    return query.offset(skip).limit(limit).all()

# Create Operation with Atomic Commit
def create_institution(db: Session, obj_in: InstitutionCreate):
    db_obj = Institution(**obj_in.model_dump())
    db.add(db_obj)          # 1. Stage in session memory
    db.commit()             # 2. Flush SQL INSERT transaction to database
    db.refresh(db_obj)       # 3. Reload generated Auto-PK (id) and timestamps
    return db_obj
```

Data Conversion & Query Lifecycle:

1. Router receives client parameters and passes active db session to CRUD function.
2. SQLAlchemy generates SQL statement: `SELECT * FROM institutions WHERE district_id = ? LIMIT ? OFFSET ?`.
3. SQLite engine executes query and returns raw binary/tabular row tuples.
4. SQLAlchemy instantiates Institution Python class instances, populating fields automatically.
5. CRUD function returns Python ORM objects back to the FastAPI router.